1.3. Phân tích độ phức tạp

YÊU CẦU: Nêu rõ độ phức tạp thời gian (time complexity) của mỗi thuật toán. Giải thích vai

trò của cấu trúc dữ liệu (ví dụ: hàng đợi ưu tiên) trong việc tối ưu hóa độ phức tạp của Thuật toán

Dijkstra.

• |V| : Số lượng đỉnh, |E| : Số lượng cạnh.

Trả lời:

1. **Thuật toán Dijkstra**

Độ phức tạp thời gian là $O(|V|^2)$ khi sử dụng ma trận kề $O\big((|V| + |E|)\backslash log\ |V|\big)$

khi sử dụng danh sách kề kết hợp hàng đợi ưu tiên Min-Heap. Thuật toán liên tục chọn đỉnh chưa xét có khoảng cách nhỏ nhất, sau đó cập nhật khoảng cách đến các đỉnh kề. Hàng đợi ưu tiên giúp tìm đỉnh gần nhất nhanh hơn, đặc biệt với đồ thị thưa. Dijkstra chỉ áp dụng đúng khi trọng số các cạnh không âm.

2. **Thuật toán Ford–Bellman (Bellman–Ford)**

Độ phức tạp thời gian là $O(|V|\backslash cdot|E|)$ Thuật toán duyệt qua tất cả các cạnh và thực hiện thư giãn tối đa \(|V|-1\) lần để tìm đường đi ngắn nhất từ một đỉnh nguồn đến các đỉnh còn lại. Thuật toán có thể xử lý cạnh âm và phát hiện chu trình âm có thể đi đến từ đỉnh nguồn.

3. **Thuật toán Floyd–Warshall (Floyd)**

Độ phức tạp thời gian là $O(|V|^3)$, độ phức tạp bộ nhớ là $O(|V|^2)$. Thuật toán sử dụng ba vòng lặp lồng nhau để lần lượt xét các đỉnh trung gian, cập nhật khoảng cách giữa mọi cặp đỉnh theo công thức**:**

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

Thuật toán xử lý được cạnh âm, nhưng không cho kết quả đường đi ngắn nhất hữu hạn nếu tồn tại chu trình âm liên quan đến đường đi đang xét.

4.Tối ưu độ phức tạp của Dijkstra

Khi không sử dụng hàng đợi ưu tiên, thuật toán phải duyệt qua các đỉnh để tìm đỉnh có khoảng cách nhỏ nhất ở mỗi bước, dẫn đến độ phức tạp $O(|V|^2)$ Khi sử dụng danh sách kề kết hợp Min-Heap, việc lấy đỉnh gần nhất và cập nhật khoảng cách được thực hiện hiệu quả hơn, giúp độ phức tạp thường đạt $O\big((|V| + |E|)log\ |V|\big)$ Vì vậy, phương pháp này đặc biệt phù hợp với đồ thị thưa, khi số cạnh nhỏ so với bình phương số đỉnh.